Grocery Store Management & WhatsApp Ordering System

Corrected Final Architecture, Features, Implementation, Security, Reliability and Production Readiness Specification

Version 2.0 | Corrected final implementation baseline | 24 October 2026

FINAL IMPLEMENTATION BASELINE

A multi-tenant microservices platform for grocery store management, WhatsApp ordering, invoices and payment links.

Grocery Store Management & WhatsApp Ordering System

Final Architecture, Features, Implementation, Security, Reliability and Production Readiness Specification

Version 2.0 | Corrected final implementation baseline | 24 October 2026

FINAL IMPLEMENTATION BASELINE

A practical specification for a secure, reliable, observable microservices-based grocery store management and WhatsApp ordering platform.

Document navigation

- 1. Purpose and status

This is the final agreed design baseline, not a claim that software can be 100% bug-free. Implementation conformance is proven only when the source code, migrations, service contracts, automated tests, security checks, staging workflows and recovery drills satisfy the acceptance criteria.

What ‘final’ means

Before coding provider-specific integration, record ADRs for the payment gateway, WhatsApp Business Platform account/webhook configuration, tax/invoice rules, customer identification, order cancellation/refund policy, deployment target, backup retention, and measurable initial load/SLO targets. These are genuine environment/business choices; they must be resolved rather than guessed.

Required pre-implementation decisions

The allowed dependency direction is: Web → Gateway → domain service APIs; domain services publish events through outbox → RabbitMQ → consumers such as Notification and Reporting. Reporting owns its read projections and consumes events; it does not query other services’ private tables. Avoid synchronous circular dependencies. Keep Catalog independent of Order; Order may call Catalog for validation/snapshots and Inventory for reservation. Inventory must not call Order synchronously to decide whether stock can be reserved. Cross-service workflows are coordinated by Order using explicit states, timeouts and compensating actions.

Dependency direction and data ownership

After confirmation, Billing generates an immutable invoice/bill snapshot and Payment creates a provider payment link for the confirmed amount. Notifications send the invoice and payment link through WhatsApp, and email when an address is available. Payment status is separate from fulfilment status. Only verified provider responses/webhooks may mark a payment successful. Duplicate webhooks, provider timeouts, refunds and reconciliation must be handled safely. The payment provider, tax configuration and production credentials are environment/ADR decisions that must be completed before live launch; they do not make these capabilities optional.

Payment and invoice policy

An incoming order starts as AWAITING_SHOPKEEPER_REVIEW; it is not a stock guarantee. When the shopkeeper confirms the final quantities and price, Order Service requests an atomic all-or-nothing inventory reservation. If reservation fails, the order cannot be confirmed as available; the shopkeeper must revise/reject it. The confirmed order snapshot is the source for invoice totals. Any later price/quantity change requires an explicit revision and a new confirmation/audit record. Stock release on cancellation is idempotent.

Order, inventory and billing policy

The initial release MUST include store registration/profile, product categories/subcategories/brands/varieties, pricing and inventory, customer identification, product search/browsing, WhatsApp ordering, shopkeeper order review/approval/rejection/change requests/alternatives, automatic itemized bill/invoice after confirmation, WhatsApp and email invoice/payment-link delivery, payment status tracking, order fulfilment/status updates, basic sales reports, low-stock/out-of-stock alerts, notifications, audit records and tenant data protection. Payment-link creation and verified payment-status updates are core requirements. Refunds are supported where applicable under the selected provider and documented business policy.

Core scope is not optional

Every registered shopkeeper has an isolated store tenant from the first release. Every tenant-owned record and query must be scoped to the authenticated store. Store IDs supplied in request bodies, query strings or headers are not trusted as proof of access. For WhatsApp webhooks, resolve the store using the verified provider business-number/account mapping. Multi-branch management for a single shopkeeper is explicitly future scope.

Tenant model: multi-tenant now; multi-branch later

This section resolves ambiguities found during review. It overrides any earlier wording in this document that conflicts with these decisions.

1A. Corrected architecture decisions — binding requirements

- 2. Product scope and users
- 3. Technology baseline
- 4. Target architecture and service boundaries
- 5. End-to-end business workflows
- 6. Domain models and state machines
- 7. Data integrity and PostgreSQL design
- 8. Synchronous and asynchronous communication
- 9. Race conditions, idempotency and failure handling
- 10. Security and cybersecurity requirements

- 11. Node.js and backend engineering practices
- 12. Networking and system-design fundamentals to apply
- 13. API standards and error handling
- 14. Frontend requirements
- 15. Observability and operations
- 16. Testing strategy and acceptance criteria
- 17. Deployment, CI/CD and environment strategy
- 18. Repository structure
- 19. Documentation deliverables
- 20. Phased implementation plan
- 21. Risk register
- 22. Core computer-engineering concepts applied
- 23. Definition of done / production release gate
- 24. Final implementation rules

1. Purpose and status

This document is the implementation baseline for the Grocery Store Management and WhatsApp Ordering System. It combines the project features and the architecture-driven, production-grade concepts discussed so the implementation can be planned, built, tested, deployed and operated consistently.

The system is a multi-tenant platform: each registered shopkeeper owns and manages their own store, while multi-branch management for one shopkeeper is a future enhancement. A microservices architecture is mandatory. The selected baseline is Node.js + TypeScript for backend services, React + TypeScript for the web application, and PostgreSQL for persistent business data. Supporting infrastructure is deliberately practical: Docker Compose for local integration, RabbitMQ for asynchronous business events, and Redis only where caching or rate limiting provides a clear benefit. Invoice generation/delivery and payment-link creation/status tracking are core requirements, not optional features.

Scope discipline: “production-grade” means correctness, security, recoverability, observability, tested deployment and documented operations. It does not mean adding Kubernetes, Kafka, a service mesh, multi-region deployment or other infrastructure before the project has a measurable need.

How to treat decisions in this specification

- MUST: required for the agreed baseline and acceptance tests.
- SHOULD: recommended unless a documented Architecture Decision Record (ADR) explains why not.
- OPTIONAL / LATER: add only when a concrete requirement, risk or measured bottleneck justifies it.

2. Product scope and users

Primary user roles

- Store owner / administrator: configures the store, users, products, prices, inventory policies, order rules and reporting access.
- Store staff / manager: maintains catalogue and stock, confirms or prepares orders, records fulfilment and handles permitted adjustments.
- Customer: browses products, places orders through the supported web/WhatsApp flow, receives status updates and views their own order information.
- System operator: deploys, monitors, backs up and troubleshoots the platform using restricted operational access.

Core product capabilities

- Product catalogue: categories, product details, images, active/inactive status, unit of measure, price and availability.
- Inventory management: stock on hand, reservations, stock adjustments, low-stock thresholds and movement history.
- Order management: cart/order creation, item and price snapshots, order lifecycle, cancellation rules and fulfilment status.
- WhatsApp ordering and notifications: receive supported customer messages or order intents through the WhatsApp Business Platform Cloud API; send order confirmations and status notifications. Exact inbound conversation flows depend on the approved WhatsApp API setup and message-template rules.
- Payments: integrate a supported payment gateway to create secure payment links for confirmed orders, track pending/successful/failed/refunded states, process verified webhooks idempotently, support refunds where applicable, and reconcile uncertain outcomes. Provider selection is an explicit pre-implementation decision, not a reason to omit payment functionality.
- Billing/invoices: generate an itemized bill after shopkeeper-confirmed order; preserve invoice/order snapshots and provide controlled download access. Invoice delivery through WhatsApp and email is core functionality.
- Reporting: sales, order counts, top-selling products, inventory status, low-stock items and basic operational metrics.
- Identity and access: sign-in, role-based access control, secure sessions/tokens and administrative audit trails.
- Operational administration: health endpoints, logs, metrics, alerts, migrations, backups, restore instructions and incident runbooks.

Explicitly out of scope unless separately approved

- Multi-region active-active, Kubernetes, service mesh, event sourcing for every domain, a data lake, advanced warehouse/ERP integration, a custom WhatsApp provider, or AI recommendations.
- Promotions, loyalty, multiple physical branches, delivery-driver tracking, tax/accounting integrations and refunds beyond the selected payment provider's supported flow must be confirmed as separate requirements.

3. Technology baseline

Layer	Baseline choice	Implementation notes
Frontend	React + TypeScript + Vite	Responsive customer and staff/admin UI; follow the project's style.md if one exists.
Backend services	Node.js + TypeScript + Express	One independently runnable service per bounded business capability; validate inputs at the boundary.
Database	PostgreSQL	ACID transactions, constraints, indexes, migrations, backups and restore testing.
ORM / SQL access	Prisma or a consistent typed SQL layer	Choose one and standardize migrations; use explicit SQL where row locking or query plans require it.
Async messaging	RabbitMQ	Durable queues, explicit acknowledgements, retry policy, dead-letter queues and event contracts.
Cache / rate limits	Redis, only where needed	Never treat cache as the source of truth for stock, payment or order state.
WhatsApp	WhatsApp Business Platform Cloud API	Verify webhook authenticity, handle retries and template/message rules.
Payments	A supported payment gateway	Create payment links for confirmed orders; verify signatures, deduplicate webhooks, track payment/refund states, and reconcile uncertain outcomes.
Runtime	Docker + Docker Compose	Repeatable local integration environment; production deployment target can evolve later.
API documentation	OpenAPI / Swagger	Versioned REST contracts, error format and examples.
Validation	Zod	Validate request bodies, query parameters, environment variables and event payloads.
Logging	Pino or equivalent structured logger	Redact secrets, tokens, personal data and payment-sensitive fields.
Testing	Vitest/Jest + Supertest + Playwright	Unit, integration, contract, end-to-end and failure-path coverage.
Load / security testing	k6 and OWASP ZAP or equivalent	Use safe test environments and documented test profiles.
Observability	OpenTelemetry + Prometheus/Grafana or equivalent	Structured logs, metrics, traces and actionable alerts.
CI	GitHub Actions or equivalent	Lint, typecheck, test, build, dependency/security checks and image build.

Pin supported versions in package manifests and container images. Avoid floating production tags. Keep secrets outside source control and document how local development secrets are created.

4. Target architecture and service boundaries

The architecture is a small set of independently deployable services with clear ownership and no circular dependency graph. The API gateway is the public entry point, but it is not the sole authorization boundary: each service validates the caller and resource/store ownership. Synchronous REST calls serve immediate reads/actions; RabbitMQ carries asynchronous business events. Each service owns its data and publishes contracts rather than exposing private tables. Tenant identity is derived from verified authentication or a verified provider-to-store mapping, never trusted from a client-supplied tenant_id.

Logical architecture

Customer / staff browser and WhatsApp webhook → API Gateway / edge controls → Identity, Catalog, Inventory, Order, Payment, Billing, Notification and Reporting services. Services use their own PostgreSQL database or clearly isolated schema/role as the initial deployment permits. Services publish and consume events through RabbitMQ. Redis is an optional supporting component, not the authoritative business store.

Service responsibilities

Service	Owns / responsibilities	Example interfaces and events
API Gateway	Public routing, request IDs, request-size limits, rate limits, coarse authentication enforcement, timeouts and consistent errors.	Routes /api/v1/*; propagates trace/correlation IDs. Avoid embedding business rules here.
Identity Service	Users, credentials, sessions/refresh tokens, roles, account status and authorization policy.	POST /auth/login, /auth/logout, /auth/refresh; UserRoleChanged event if needed.
Catalog Service	Products, categories, descriptions, images, sale status and current catalogue metadata.	GET /products; POST/PATCH /admin/products; ProductCreated/ProductUpdated.
Inventory Service	Stock balances, reservations, stock movements, low-stock thresholds and inventory adjustments.	GET /inventory; POST /reservations; StockReserved/StockReservationFailed/StockReleased.
Order Service	Order creation, order item snapshots, order state machine, cancellation request and order history.	POST /orders; GET /orders/{id}; OrderPlaced/OrderCancelled/OrderReady.
Payment Service	Payment attempts, provider references, webhook verification, payment/refund state, payment links and reconciliation.	POST /payments; provider webhook endpoint; PaymentAuthorized/PaymentFailed/PaymentRefunded.
Billing Service	Invoice/receipt generation and immutable invoice data tied to the order snapshot.	GET /invoices/{id}; InvoiceIssued.
Notification Service	WhatsApp/email delivery, templates, delivery attempts, retries and provider callback processing.	Consumes OrderPlaced/OrderStatusChanged/Payment*; NotificationSent/NotificationFailed.
Reporting Service	Read-optimized reports and projections; does not become the owner of other services' write models.	GET /reports/sales; consumes domain events to update reporting projections.

Data ownership rules

- A service may read/write its own tables. Another service must not directly modify that service's tables.
- Cross-service reads use documented APIs or asynchronously maintained reporting projections. Do not create cross-service foreign keys.
- For a first release, services may use one PostgreSQL server but should have separate databases or schemas and separate least-privilege database roles. The stronger production target is separately managed databases when deployment and operational capacity justify it.
- Do not use a shared database transaction across services. Use local ACID transactions plus explicit distributed workflow/state handling.

Avoid over-fragmentation

Start with the service boundaries above only if they map to actual project requirements. Billing can be a module within Order initially if there is no separate invoicing workflow; Reporting can begin as a consumer/projection. Keep a service separate when it needs independent ownership, scaling, security or release cadence—not just to increase the service count.

5. End-to-end business workflows

A. Browse and place an order

1. Customer requests catalogue through the gateway; Catalog returns active products and prices.
1. Customer submits an order with product IDs, quantities, delivery/pickup details and an idempotency key.
1. Order Service validates the request and fetches/validates required product data through the Catalog API or a documented snapshot strategy. Client-supplied prices are never trusted.
1. Order Service creates a pending order and immutable item/price/name/unit snapshots in a local transaction. It coordinates stock reservation with Inventory using a clearly defined workflow.
1. Inventory atomically checks and reserves available stock. A failed reservation causes the order workflow to fail or enter a controlled review state; it must not silently oversell.
1. After the order and reservation outcome are recorded, OrderPlaced is published through the transactional outbox. Notification Service sends confirmation asynchronously.
1. Order status progresses through explicit states. Staff actions are authorized, audited and validated against allowed transitions.

B. WhatsApp ordering

1. WhatsApp sends a webhook to a dedicated inbound endpoint. Verify the provider signature/token as documented by Meta before processing the payload.
1. Persist the provider message ID / webhook event ID with a uniqueness constraint so duplicate deliveries do not create duplicate orders or replies.
1. Normalize the message into a supported command or conversational step. Ask for missing information; do not infer an order from ambiguous text.
1. Use the same Order and Inventory APIs as web orders. Do not implement a separate inventory/order rule set inside the webhook handler.
1. Acknowledge quickly, then send longer-running updates asynchronously. Handle provider timeouts, retries and template requirements.

C. Payment workflow (core requirement)

1. Create a local payment attempt with a unique internal ID and provider idempotency key where supported.
1. Call the provider with a strict timeout. A timeout means the result may be unknown; query/reconcile before creating a second logical payment.
1. Verify webhook signature against the raw request body and configured secret; deduplicate provider event IDs.
1. Apply a valid state transition in a local transaction and publish PaymentAuthorized/PaymentFailed via outbox.

1. Do not mark an order paid solely because the browser says payment succeeded. Use verified provider response/webhook and reconciliation.

D. Cancellation and stock release

1. Validate that the order state allows cancellation and record the request idempotently.
1. Release stock exactly once using a reservation identifier and an idempotent Inventory operation.
1. If payment has already completed, trigger the supported refund/compensation workflow rather than pretending the payment never happened.
1. Publish status changes through outbox and notify the customer asynchronously.

6. Domain models and state machines

Minimum data concepts

- User: id, normalized email/phone, credential/session metadata, status, roles and timestamps.
- Product: id, SKU, name, description, category ID, unit, current price, currency, active flag, image reference and timestamps.
- Inventory balance: product/SKU reference, available quantity, reserved quantity or an equivalent consistent representation, version and timestamps.
- Inventory movement: immutable movement ID, product ID, quantity delta, reason, actor/source, related order/reservation and timestamp.
- Order: id, customer reference, status, currency, totals, fulfillment details, idempotency reference and timestamps.
- Order item: order ID, product ID reference, product name/SKU/unit/price snapshot, quantity, tax/discount snapshot if applicable and line total.
- Reservation: reservation ID, order ID, items, expiry/state and idempotency key.
- Payment attempt: internal ID, order ID, provider reference, amount/currency, status, idempotency key and timestamps.
- Outbox/inbox: event ID, aggregate ID, event type/version, payload, occurred/published/processed timestamps, retry count and status.
- Audit log: actor, action, target, timestamp, request/correlation ID and minimal before/after details without secrets.

The core order workflow is: RECEIVED/AWAITING_SHOPKEEPER_REVIEW → CHANGE_REQUESTED (optional) → CONFIRMED → PREPARING → READY_FOR_PICKUP or DISPATCHED → COMPLETED/DELIVERED. REJECTED and CANCELLED are terminal outcomes subject to policy. Inventory is reserved atomically when the final order is confirmed, not merely because an unreviewed request arrived. If the business later chooses reservation-at-submission, that is a deliberate ADR with expiry and customer messaging.

Suggested order state machine

PENDING → CONFIRMED → PREPARING → READY → COMPLETED. PENDING may transition to CANCELLED or FAILED. CONFIRMED/PREPARING cancellation must follow an explicit policy. A payment-enabled implementation may model payment status separately rather than mixing every payment state into the order state.

Suggested payment state machine

CREATED → PENDING_PROVIDER → AUTHORIZED/PAID or FAILED. Refunds are separate states/records such as REFUND_PENDING → REFUNDED/REFUND_FAILED. Transitions must be validated and replay-safe.

State machine rules

- Every state transition is checked on the server; never accept arbitrary status values from a client.
- Record who/what caused sensitive transitions and make repeated requests safe.
- Use explicit timestamps and reason codes for failure/cancellation, and define how stale pending states are reconciled.

7. Data integrity and PostgreSQL design

- Use ACID transactions for changes that belong to one service and require atomicity.
- Enforce invariants with database constraints: primary/foreign keys inside a service, unique provider event IDs, unique idempotency keys in the proper scope, non-negative quantities where appropriate and valid currency/amount constraints.
- Represent money using integer minor units (for example paise) or PostgreSQL NUMERIC with a documented precision. Never use binary floating-point for authoritative money calculations.
- Store order item and invoice snapshots so later product price/name changes do not rewrite historical orders.
- Add indexes based on actual query patterns: product active/category/name search, order customer/status/created_at, inventory SKU, payment provider reference and event/outbox status. Validate with EXPLAIN (ANALYZE, BUFFERS) using representative data.
- Use migrations checked into version control; review migration locking and rollout impact before production.
- Configure connection pools deliberately. A connection pool per service can multiply into excessive connections; budget total connections across all replicas.
- Understand MVCC, isolation levels, row locks, deadlocks and transaction retries. Keep transactions short and never hold a DB transaction open while waiting for a remote API.
- Backups are not sufficient until restore is tested. Document RPO/RTO targets, backup retention, encryption and restore validation.

Stock reservation correctness

The inventory service is the authority for available stock. Use an atomic conditional update or row-level lock inside a transaction, for example update stock only when available_quantity >= requested_quantity and check the affected row count. For multi-item reservations, use a consistent lock order, validate all items and commit all-or-nothing within Inventory's database. Never use "read quantity, then update later" without concurrency protection.

Deadlocks and retries

Lock rows in a stable order, keep transactions short, set sensible statement/transaction timeouts and retry only known transient failures with bounded attempts. A retry must not duplicate the business operation.

8. Synchronous and asynchronous communication

Use REST when

- The caller needs an immediate result, such as reading a product, authenticating, or requesting an order action.
- The interaction needs request validation, clear status codes, an explicit timeout and a documented error contract.

Use RabbitMQ events when

- A completed business action must trigger notifications, reporting updates or other work that can happen after the original request.
- The consumer can process the event eventually and safely retry it.

Event contract requirements

- Every event includes event_id, event_type, schema_version, occurred_at, producer, aggregate_id and correlation/trace ID; payload fields are explicitly documented.
- Version event schemas compatibly. Do not publish credentials, full payment data or unnecessary personal data.
- Consumers acknowledge only after durable processing. Use inbox/processed-event uniqueness to prevent duplicate side effects.
- Use durable queues and messages, publisher confirms where appropriate, bounded retries, dead-letter queues and operational dashboards.
- Design for at-least-once delivery; do not assume exactly-once delivery.

Transactional outbox

When a service updates business state and must publish an event, write the business change and outbox row in the same local database transaction. A publisher worker sends pending rows to RabbitMQ and marks them published only after broker confirmation. If the process crashes, the row remains available for retry. Consumers use an inbox table or unique event ID to make processing idempotent.

Saga / compensation

A multi-service workflow cannot rely on one distributed SQL transaction. Model the steps and failure outcomes explicitly. For example: create pending order → reserve stock → authorize payment if enabled → confirm order. If payment fails, release stock and fail/cancel the order. Each compensation must itself be idempotent, observable and retryable.

9. Race conditions, idempotency and failure handling

Idempotency

- Accept an Idempotency-Key for order/payment creation where repeated requests could create duplicate effects.
- Store key, actor/tenant scope, request hash, status and result reference under a unique constraint. A repeated key with the same request returns the original logical result; a different request body with the same key is rejected.
- Make stock reservation/release, refund initiation, notification processing and webhook handling replay-safe.

Race-condition scenarios to test

- 100 concurrent orders attempt to buy 10 available units: no more than 10 units are reserved.
- Two staff users update the same stock balance: changes are serialized or conflict is detected; no silent lost update.
- Two duplicate webhooks arrive simultaneously: only one business transition occurs.
- A client retries after a timeout: no duplicate order or payment is created.
- A cancellation and payment confirmation race: final state follows a documented transition policy and reconciliation repairs uncertain cases.

Resilience controls

- Set connection, request and dependency timeouts. Use per-dependency limits; do not allow requests to wait indefinitely.
- Use bounded retries with exponential backoff and jitter only for transient, retryable failures. Avoid retry storms and retries at every layer.
- Use circuit breakers or equivalent failure isolation for unstable remote dependencies where justified.
- Use bounded concurrency and backpressure for queues and CPU-heavy tasks. Apply message prefetch and worker concurrency limits.
- Route exhausted messages to a dead-letter queue with alerting and a documented replay procedure.
- Add reconciliation jobs for uncertain payment outcomes, expired reservations and stuck pending orders.
- Implement graceful shutdown: stop accepting new work, mark readiness false, finish or safely abandon in-flight work, close consumers and pools, then exit within a deadline.

10. Security and cybersecurity requirements

Identity and authorization

- Use secure password hashing (Argon2id or appropriately configured bcrypt), rate-limited authentication and secure account recovery.
- Use secure, HttpOnly, SameSite cookies for browser sessions when using cookie-based auth; protect state-changing requests against CSRF. If JWTs are chosen, define expiry, refresh, revocation and storage rules; do not store long-lived bearer tokens in localStorage by default.
- Enforce authorization on every server-side resource access. Validate both role and resource ownership; hiding a button in React is not security.
- Prevent Broken Object Level Authorization (BOLA): a customer must not read/change another customer's order by guessing an ID.
- Apply least privilege to staff roles, service accounts, database users and deployment credentials.

Input and application security

- Validate all external inputs with Zod and reject unknown or malformed fields where appropriate.
- Use parameterized queries/ORM-safe APIs; avoid string-built SQL.
- Prevent XSS through context-safe output and safe rendering; do not render untrusted HTML.
- Use CSRF protection when cookie-based authentication is used; configure CORS to explicit trusted origins.
- Defend against SSRF if the server fetches user-controlled URLs or images: restrict schemes, destinations, redirects and private-network access.
- Set request body and upload size limits, content-type checks, file type validation and safe image storage rules.
- Set security headers, HTTPS, HSTS at the appropriate edge, secure cookies and safe error messages.

API, webhook and infrastructure security

- Rate-limit login, OTP/recovery, order creation, search and public webhooks according to abuse risk. Combine IP and user/account limits where appropriate.
- Verify WhatsApp/payment webhook signatures or authentication exactly as required by the provider; verify raw body when the signature scheme requires it.
- Store secrets in environment-specific secret management; never commit .env files, provider tokens or signing keys. Rotate secrets and document rotation.
- Redact passwords, tokens, session IDs, payment details and unnecessary personal information from logs.
- Use dependency scanning, secret scanning, container image scanning, static analysis and periodic dynamic security tests.
- Threat-model abuse cases: unauthorized order access, stock manipulation, forged webhook, replay, brute force, price tampering, privilege escalation, queue poisoning and denial of service.

Audit and privacy

- Audit administrative actions, stock adjustments, refunds, role changes and sensitive order-state changes.
- Collect only necessary personal data; define retention, access, deletion and backup-retention policies appropriate to applicable law and business requirements.
- Restrict access to logs and exports. Avoid putting personal data in event payloads unless the consumer needs it.

11. Node.js and backend engineering practices

- Understand the event loop, microtasks/promises, libuv thread pool, asynchronous I/O and the difference between I/O-bound and CPU-bound work.
- Do not block the event loop with large synchronous loops, synchronous filesystem operations or expensive JSON processing on request paths.
- Measure event-loop delay and request latency. Use worker threads or a separate worker service for measured CPU-heavy work, not as a default for every task.
- Use streams and backpressure for large files/exports rather than loading everything into memory.
- Handle rejected promises and errors consistently. Every route and background worker must have a defined error path.
- Use process signal handling and graceful shutdown; ensure health/readiness endpoints reflect dependency and startup state correctly.
- Configure HTTP keep-alive, payload limits, connection pools, timeouts and queue worker concurrency intentionally.
- Use structured logs and propagate request/correlation IDs across REST calls and events.
- Keep configuration validated at startup; fail fast for missing required environment variables without exposing secret values.
- Separate route/controller, application/use-case, domain/business rule and infrastructure/data-access layers where it improves testability. Avoid a large generic abstraction framework.

12. Networking and system-design fundamentals to apply

- HTTP semantics: methods, status codes, headers, caching, content negotiation and safe/idempotent request behavior.
- TCP/IP, DNS, TLS certificates, connection setup, keep-alive, reverse proxies and load balancing.
- Timeout budgets: an upstream deadline must leave enough time for downstream work and response handling.
- Backpressure and load shedding: reject or defer work safely when queues, pools or dependencies are saturated.

- Caching: define cache keys, TTL, invalidation and stale-data tolerance. Never use cache alone for authoritative stock or payment decisions.
- Availability versus consistency: use strong consistency for stock/payment invariants and eventual consistency for notifications/reporting.
- Capacity planning: estimate request rate, average payload, database queries per request, connection count, queue backlog and storage growth.
- Failure domains: a notification provider outage should not make a successfully placed order disappear.
- SLOs and error budgets: define what the system promises, measure it and prioritize work based on observed reliability.

13. API standards and error handling

- Use versioned paths such as /api/v1/... and document every endpoint in OpenAPI.
- Use consistent success response shapes and a consistent problem-details error structure (RFC 9457 / problem details style). Include code, title, status, safe detail, request_id and validation errors where appropriate.
- Validate path/query/body/header inputs; do not trust client totals, roles, product availability or payment status.
- Use correct HTTP status codes: 400/422 for invalid input, 401 unauthenticated, 403 unauthorized, 404 missing/hidden resource, 409 conflict, 429 rate-limited, 5xx server/dependency failure.
- Set pagination, filtering and sorting limits; use stable ordering for paginated lists.
- Define timeout and retry behavior per endpoint. Do not automatically retry non-idempotent requests without idempotency protection.
- Keep API and event contracts in shared versioned packages or a dedicated contracts directory; generated types do not remove the need for runtime validation.

14. Frontend requirements

- Build separate customer and staff/admin experiences with route-level access checks for usability, while relying on backend authorization for actual security.
- Provide clear loading, empty, validation, error, retry and success states.
- Prevent duplicate submissions in the UI but also enforce idempotency on the server.
- Display stock/price as potentially stale until confirmed by the server; the server is authoritative at checkout.
- Use accessible labels, keyboard navigation, responsive layouts and understandable form validation.
- Do not expose secrets, internal service URLs, database details or provider credentials in frontend environment variables.
- Keep API client logic centralized and handle authentication expiry, request IDs and structured API errors consistently.
- Follow the project's established style.md/design system if supplied; do not invent a different visual system without approval.

15. Observability and operations

Logs, metrics and traces

- Structured JSON logs include timestamp, level, service, environment, request_id/correlation_id, route, duration and safe error metadata.
- Metrics include request rate, error rate, latency percentiles, event-loop delay, DB pool saturation, query duration, queue depth/age, retry count, dead-letter count, stock reservation failures, order/payment state age and notification failure rate.
- Distributed traces connect gateway → service → database/message workflow where instrumentation supports it.
- Never use high-cardinality or personal identifiers as metric labels.

Health endpoints

- Liveness: process is alive and not irrecoverably stuck.
- Readiness: service can accept traffic based on startup and critical dependency status. Avoid making every transient optional dependency failure restart the whole service.
- Startup checks validate configuration and required schema/migration state without dumping secrets.

Alerts and runbooks

- Alert on sustained 5xx/latency breaches, queue backlog/oldest-message age, dead-letter growth, DB pool saturation, repeated webhook failures, stuck pending payments/orders and backup failures.
- Each actionable alert links to a runbook with impact, first checks, safe mitigation, escalation and recovery verification.
- Define RPO (acceptable data loss) and RTO (acceptable recovery time) with the business owner; do not claim a target that has not been tested.

16. Testing strategy and acceptance criteria

Required test layers

- Unit tests: domain rules, state transitions, validation, money calculations and authorization policy.
- Integration tests: PostgreSQL constraints/transactions, migrations, RabbitMQ consumers, outbox/inbox and provider adapters using controlled test doubles or sandbox environments.
- Contract tests: REST and event schema compatibility between services.

- End-to-end tests: sign in, browse, place order, reserve stock, staff status update, notification and invoice flow.
- Security tests: BOLA, role escalation, injection, CSRF where applicable, XSS, rate limits, webhook signature/replay and secrets exposure.
- Load tests: normal load, burst load, slow dependency, queue backlog and recovery after load.
- Failure tests: broker unavailable, provider timeout, worker crash after DB commit, duplicate event, dead-letter replay, DB connection interruption and graceful shutdown.

Minimum release acceptance tests

1. With 10 units available and 100 concurrent attempts to reserve one unit, at most 10 reservations succeed; stock never becomes negative.
1. Submitting the same order request with the same idempotency key repeatedly creates one logical order and returns a consistent result.
1. Delivering the same payment webhook multiple times, including concurrently, causes one valid business effect.
1. A customer cannot read, cancel or modify another customer's order by changing an identifier.
1. A staff user without the required permission cannot adjust stock, change roles or issue refunds.
1. If RabbitMQ is unavailable after a database commit, the outbox retains the event and publishes it after recovery without losing the business action.
1. A consumer crash/retry does not duplicate invoices, stock releases or notifications beyond the documented delivery semantics.
1. Payment provider timeouts are treated as unknown until reconciled; the system does not blindly create duplicate charges.
1. A failed deployment can be rolled back or recovered using documented steps; database migration compatibility is considered.
1. A backup can be restored into an isolated environment and the restored data passes validation.
1. Rate limits, payload limits, authentication, authorization, webhook verification and secret redaction have automated regression tests.
1. The system can shut down gracefully without accepting new work indefinitely or silently dropping acknowledged messages.

Tenant isolation acceptance: create two stores with distinct shopkeepers and customers. Attempt cross-tenant reads/writes against products, categories, inventory, customers, orders, invoices, payment records, reports and settings using changed IDs, forged tenant fields and direct service calls. Every unauthorized attempt must be rejected and logged safely.

WhatsApp tenant resolution acceptance: a valid webhook for Store A can only affect Store A; an invalid signature or unknown provider business-number mapping cannot create or alter an order.

Payment-link acceptance: a confirmed order produces one logical payment request for the correct invoice amount; duplicate create requests and webhook deliveries do not duplicate payment records or notifications.

Performance acceptance

Set numerical latency/throughput targets after the expected user count, deployment size and workload are agreed. Use a documented k6 profile and report p50/p95/p99, error rate, throughput, CPU/memory, DB pool use and queue age. Do not claim production capacity based only on a successful local smoke test.

17. Deployment, CI/CD and environment strategy

- Maintain local, test/staging and production configuration separately. Production secrets must not be copied into development.
- Dockerize services with pinned base images, non-root users where practical, minimal runtime images and health checks.
- Use Docker Compose for reproducible local integration: services, PostgreSQL, RabbitMQ, optional Redis and observability components.
- CI must run formatting/lint, typecheck, unit tests, integration/contract tests where practical, build, migration validation, dependency/secret checks and container scanning.
- Use immutable image tags/digests and a documented deployment process. Do not let coding agents or scripts start servers without explicit instruction.
- Apply database migrations as a controlled deployment step. Prefer backward-compatible expand/migrate/contract changes when multiple service versions may run simultaneously.
- Use readiness checks and a controlled rollout; verify key synthetic flows after deployment.
- Keep rollback instructions and note when a database change is not safely reversible.
- Document resource limits, environment variables, ports, health checks, volumes, backup locations and restore commands.

Kubernetes and service mesh are later options, not baseline requirements. Add them only if deployment scale, availability targets or team operations justify their cost.

18. Repository structure

grocery-platform/

apps/

web/

api-gateway/

identity-service/

catalog-service/

inventory-service/

order-service/

payment-service/ # only if online payments are in scope

billing-service/ # can begin within Order if scope is small

notification-service/

reporting-service/

packages/

api-contracts/

event-contracts/

configuration/

observability/

testing/

infrastructure/

docker/

monitoring/

deployment/

docs/

architecture/

diagrams/

adr/

security/

api/

events/

testing/

runbooks/

scripts/

docker-compose.yml

package.json

README.md

Keep the original project brief/reference documents outside the implementation folder if that is the agreed workflow. Put only approved, distilled requirements and architecture artifacts in docs/.

19. Documentation deliverables

- README.md: prerequisites, installation, environment setup, commands, test commands and troubleshooting.
- PRD.md: actors, use cases, assumptions, exclusions and acceptance criteria.
- ARCHITECTURE.md: system context, containers, trust boundaries, communication and deployment topology.
- SERVICE-BOUNDARIES.md: ownership, APIs, events, data and dependency rules for each service.
- DATABASE-DESIGN.md: schemas, constraints, indexes, migrations, retention and backup policy.
- API OpenAPI specs and event schemas with versioning policy and examples.
- THREAT-MODEL.md: assets, trust boundaries, abuse cases, controls and residual risks.
- ADRs: important decisions with context, alternatives, decision, consequences and revisit conditions.
- TEST-STRATEGY.md: test layers, fixtures, concurrency/failure scenarios and release gates.
- SLO-AND-MONITORING.md: service objectives, dashboards, alerts and metric definitions.
- RUNBOOKS: deploy/rollback, restore, queue backlog, dead-letter replay, provider outage, stuck payment/order and secret rotation.
- WhatsApp/provider setup guide: webhook registration, verification, token rotation, message templates, sandbox/test numbers and troubleshooting.

20. Phased implementation plan

Phase 0 — Requirements and architecture lock

- Confirm user roles, tenant/store registration rules, pickup/delivery options, payment provider, cancellation/refund policy, tax/invoice rules, and WhatsApp conversation scope. Tenant isolation, payment links, invoice generation/delivery, and payment-status tracking are already core requirements and must not be removed.
- Create PRD, C4 diagrams, service boundaries, data ownership, state machines, API/event contracts, threat model and initial ADRs.
- Define measurable initial SLO/load assumptions and local developer workflow. No implementation starts until decisions and acceptance criteria are reviewed.

Phase 1 — Repository and platform foundation

- Create workspace, strict TypeScript configuration, lint/format, environment validation, structured logging, request IDs, error middleware, OpenAPI skeleton, CI and Docker Compose.
- Implement liveness/readiness, migrations, test setup and service templates. Verify each service starts only when explicitly run by the developer.

Phase 2 — Identity and catalogue

- Implement authentication/session lifecycle, roles and server-side authorization; seed safe demo users through a controlled script.
- Implement categories/products, search/filter/pagination, price and active status; add constraints and indexes.

Phase 3 — Inventory correctness

- Implement stock balances, movement ledger, adjustments, reservation/release APIs and low-stock events.
- Pass concurrency tests before integrating order placement.

Phase 4 — Orders and workflow reliability

- Implement order snapshots, state machine, idempotency, cancellation, reservation coordination, outbox/inbox, event contracts and reconciliation.
- Test duplicate requests, crash points, broker outage and recovery.

Phase 5 — WhatsApp and notifications

- Implement verified webhook endpoint, inbound deduplication, supported order-intent flow, templates, async delivery, retries and delivery tracking.
- Keep provider credentials and templates configurable by environment.

Phase 6 — Payments and billing (required core workflow)

- Integrate provider sandbox, idempotent payment attempts, signed webhook processing, reconciliation, refund policy and invoice/receipt generation.
- Do not go live until duplicate/replayed webhook and unknown-timeout scenarios pass.

Phase 7 — Reporting and staff UI

- Build sales/order/inventory reports from owned data or event-fed projections; define freshness expectations.
- Complete staff workflows, audit trails, filters, exports and access control.

Phase 8 — Production hardening

- Complete security review, load/failure testing, alerting, runbooks, backup/restore drill, migration/rollback review and dependency/container scanning.
- Run end-to-end release acceptance tests in staging; fix all critical/high findings before production.

Phase 9 — Controlled launch and improvement

- Launch with documented support/rollback plan, monitor SLOs, review errors and queue health, and prioritize based on real evidence.
- Add Redis caching, autoscaling, Kubernetes, service mesh or more advanced analytics only when metrics and requirements justify them.

21. Risk register

Risk	Impact	Required mitigation
Overselling due to concurrent requests	Inventory inconsistency and customer disputes	Atomic reservation, DB constraints/locks and concurrency tests.
Duplicate order/payment from retries	Double charges or duplicate fulfilment	Idempotency keys, provider idempotency, unique event IDs and reconciliation.
Lost event after database commit	Missing notifications/reporting/workflow steps	Transactional outbox, durable broker, monitoring and replay procedure.
Forged/replayed webhook	Fraudulent state change	Signature verification, event deduplication, timestamp/replay controls where supported.
Cross-customer or cross-tenant data access	Privacy breach	Per-resource authorization and BOLA regression tests.
Service/database coupling	Fragile deployments and data corruption	Data ownership, no cross-service writes/FKs, versioned APIs/events.
Provider/broker outage	Delayed customer updates or stuck workflows	Timeouts, retries, DLQ, outbox, reconciliation and runbooks.
Unrestored backup	Long outage or data loss	Scheduled backup plus recurring restore drills and measured RPO/RTO.
Excessive microservices complexity	Slow delivery and operational burden	Small bounded service set, shared platform conventions, split only for a reason.
Secrets or personal data in logs	Security/privacy incident	Central redaction, secret scanning, log access controls and tests.

22. Core computer-engineering concepts applied

- DBMS: ACID, normalization, constraints, indexes, transactions, MVCC, isolation, deadlocks and query planning are applied to orders and stock.
- Operating systems: processes, threads, scheduling, memory, signals, resource limits and deadlocks are applied to Node.js runtime behavior and graceful shutdown.
- Computer networks: TCP/IP, DNS, HTTP, TLS, timeouts, proxies, load balancing and connection management are applied to service communication.
- Data structures and algorithms: maps/sets for in-process lookups, heaps/top-k for reports when appropriate, pagination, indexing and complexity analysis; do not load unbounded datasets into memory.
- Distributed systems: partial failure, retries, duplicate delivery, eventual consistency, idempotency, sagas, outbox/inbox, leaderless assumptions and reconciliation.
- Cybersecurity: authentication, authorization, least privilege, input validation, threat modeling, secure secrets and defense in depth.
- Software engineering: modularity, code review, test pyramid, CI/CD, semantic/versioned contracts, ADRs and maintainable documentation.
- Performance engineering: profiling, latency percentiles, throughput, queue depth, DB pool saturation, backpressure and load testing.

23. Definition of done / production release gate

The system is not considered ready for production until all applicable items below are complete and evidenced. This specification reduces ambiguity but cannot guarantee defect-free software; production readiness must be demonstrated by code review, automated tests, security review, restore drills and staging acceptance.

- All agreed product requirements and role permissions are traceable to tests.
- Service boundaries, data ownership, API contracts and event contracts are documented and reviewed.
- No known critical/high security findings remain without explicit risk acceptance.
- Inventory concurrency and order/payment idempotency tests pass.
- Outbox/inbox, retry/DLQ and reconciliation paths are tested under failure.
- Authentication, per-resource authorization, webhook verification and audit logging are tested.
- Migrations are reproducible; indexes and connection pools are reviewed.
- CI is green; container images and dependencies are scanned; configuration and secrets are validated.
- Dashboards and actionable alerts exist for core service health, queues, database and provider integration.
- Backups and a restore drill are completed; RPO/RTO are documented.
- Deployment, rollback, provider outage, dead-letter replay and incident runbooks are usable by someone other than the original developer.
- Performance test report documents workload, environment, results, bottlenecks and limitations.
- Staging end-to-end acceptance passes, and a launch/rollback owner is identified.

24. Final implementation rules

1. Build the simplest design that meets the required feature and reliability goals; record consequential decisions in ADRs.
1. Do not let frontend input, caches, messages or provider callbacks bypass authoritative service validation.
1. Never assume a network call succeeded or failed solely because it timed out; model unknown outcomes and reconcile.
1. Treat every event and webhook as potentially duplicated, delayed or out of order.
1. Use local transactions for local invariants; use outbox/inbox and sagas for cross-service workflows.
1. Test concurrency and failure cases before declaring order and inventory flows complete.
1. Keep logs useful but free of secrets and unnecessary personal data.
1. Do not add infrastructure solely for appearance. Introduce it when a measured need and operating plan exist.
1. Do not start servers or make live provider calls automatically during implementation; run them only when explicitly requested and in the intended environment.
1. Use this document as the baseline, but record any change to a MUST-level requirement through a reviewed ADR and update the tests/docs.